Quantum Machine Learning for Join Order Optimization using Variational Quantum Circuits

Tobias Winker
Univeristy of Lübeck
Lübeck, Germany
winker@ifis.uni-luebeck.de

Umut Çalıkyılmaz
Univeristy of Lübeck
Lübeck, Germany
calikyilmaz@ifis.uni-luebeck.de

Le Gruenwald
University of Oklahoma
Norman, USA
ggruenwald@ou.edu

Sven Groppe
Univeristy of Lübeck
Lübeck, Germany
groppe@ifis.uni-luebeck.de

ABSTRACT

The optimization of queries speeds up query processing in databases. One of the most time-consuming tasks in query processing is the join operation, where the order of the joins plays a crucial role in determining the number of tuples to be processed for intermediate results, and hence, the overall processing costs. In this paper, we use a variational quantum circuit (VQC) to create a hybrid classical-quantum machine learning algorithm to predict efficient join orders by learning from past join orders. We develop an encoding of the join order problem using a low number of qubits. We show that VQCs with filtering of cross joins outperform the classical dynamic programming optimizer of PostgreSQL with a 2.7% faster execution time.

CCS CONCEPTS

• **Hardware → Quantum computation**; • **Computing methodologies** → *Machine learning*; • **Information systems → Query optimization**.

KEYWORDS

Quantum computing, Machine learning, Database optimization

ACM Reference Format:
Tobias Winker, Umut Çalıkyılmaz, Le Gruenwald, and Sven Groppe. 2023. Quantum Machine Learning for Join Order Optimization using Variational Quantum Circuits. In *The International Workshop on Big Data in Emergent Distributed Environments (BiDEDE '23), June 18, 2023, Seattle, WA, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3579142.3594299

1 INTRODUCTION

Query optimization is a crucial component of every relational database management system since the prototype System R [26]. The join operation is generally the most demanding operation for a query, so it is very important to determine the optimal join orders of the database tables when optimizing queries. The original method proposed to solve the join order optimization problem combines dynamic programming with some heuristics [31]. However, the size of the problem required researchers to seek more efficient methods to solve it. Various machine learning approaches were proposed to accomplish that task, some of which are cited below. In this paper, to further extend these studies, we propose a quantum machine learning approach for join order optimization.

One use of machine learning is to estimate the runtime of a query or its cardinality [2, 10, 16] using Support Vector Machines [12], PQR Trees [11] or neural networks [41]. The query optimizer can use this to choose the best query execution plan. Machine learning for cardinality estimation was implemented in PostgreSQL [37]. Another approach is to predict the optimal join order directly using reinforcement learning [4, 13, 20, 35, 38], which has the benefit of a short runtime of the query optimizer, while achieving good join orders.

Quantum approaches for query optimization exist, which model the join order problem as a quadratic unconstrained binary optimization (QUBO) problem [8, 22, 29, 34]. There are also many works on quantum machine learning [3, 15, 27]. Quantum Support Vector Machines can provide an exponential speedup over classical Support Vector Machines [27]. Quantum machine learning methods can potentially learn on fewer data points than classical methods alone [3]. The authors of [15] identify the properties of data sets that have a potential quantum advantage in learning tasks. A common approach for quantum machine learning is variational quantum circuits (VQC) [5, 6, 33] which were developed for problems like the Cart-Pole problem [5] and Blackjack [18]. VQC can achieve the same results as neural networks with fewer parameters [7].

In this paper, we develop a quantum machine learning approach using VQC for join order optimization. We then experimentally compare it with classical machine learning for SQL queries using PostgreSQL. The experimental results show the superiority of our approach using quantum machine learning for join order optimization.

Our main contributions are the following:

- A method for encoding a join order problem into a quantum state and decoding a join order from a quantum state.
- An improvement of this approach by eliminating cross joins with different methods.

BiDEDE '23, June 18, 2023, Seattle, WA, USA

ACM ISBN 979-8-4007-0093-4/23/06.
https://doi.org/10.1145/3579142.3594299

- A join order optimizer using VQCs, which is tightly integrated into the PostgreSQL database management system, such that the full SQL language is supported by the overall system.
- An experimental evaluation and analysis showing that VQCs perform better than classical optimizers of PostgreSQL.

The rest of the paper is organized as follows. Section 2 summarizes the background concepts of quantum computing, VQCs, and data encoding. In Section 3 we introduce an encoding for the join order problem and introduce a learning algorithm using VQCs for it. Section 4 contains the details of our implementation of this approach. Section 5 presents the experimental model with and without cross join elimination and the comparison to the join order optimizer of PostgreSQL. Finally, Section 6 concludes the paper and presents our future work.

2 QUANTUM COMPUTING

In this section, we provide a short background on quantum computing. First, we briefly introduce quantum mechanics and quantum computing. Then we present the quantum circuit model to explain VQCs and their use for learning algorithms. Finally, we describe some encoding schemes that could be used for quantum machine learning.

2.1 Quantum Mechanics and Quantum Computing

Quantum mechanics is a theory used to forecast the behavior of very small particles. Unlike classical particles, quantum particles can assume states that are superpositions of multiple classical states. However, such a particle collapses to one of these classical states when measured [28]. Similarly, a qubit, which is the most basic unit of quantum information, can be in a superposition of the states of a classical bit. The state of a qubit is represented by a two-dimensional normalized vector as follows:

$$|\psi\rangle = a_0\,|0\rangle + a_1\,|1\rangle \equiv \begin{pmatrix} a_0 \\ a_1 \end{pmatrix}, \tag{1}$$

where a_0 and a_1 are complex numbers satisfying the condition $|a_0|^2 + |a_1|^2 = 1$. When a qubit is represented by the state given in equation (1), the probability of outcomes 0 and 1 will be $|a_0|^2$ and $|a_1|^2$, respectively.

Similarly, a system of n qubits is represented by a 2^n-dimensional normalized vector. A multiple-qubit system can be in a state that cannot be described as a combination of single-qubit systems. When two qubits are in such a state, they are not independent of each other, and measuring one of them will also fix the state of the other one. This phenomenon is called entanglement and is very important to exploit quantum supremacy. We refer to [23] for further reading.

2.2 Variational Quantum Circuits (VQCs)

The most common way to show the mechanism of a quantum algorithm is the quantum circuit model [23]. In this model, each qubit is represented by a wire, and quantum gates are placed onto these wires in a required order. Examples of this model are given in Fig. 1 and Fig. 2. Quantum gates are unitary operators that can alter the state of a single qubit or multiple qubits. A common 2-qubit gate

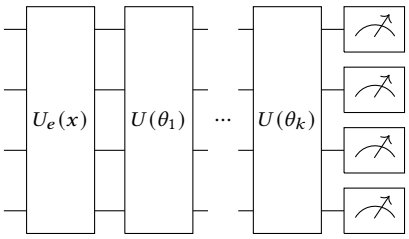

Figure 1: Example architecture of a VQC with k layers. The encoding is an operator depending on the input x. Each of the calculation layers is an operator which depends on a different parameter vector θ_i.

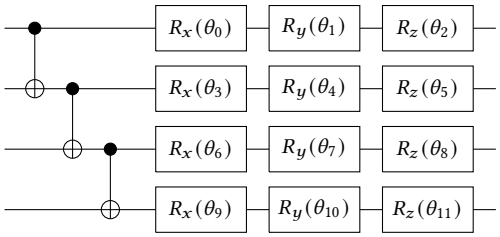

Figure 2: Example for a possible layer of a VQC using CNOT gates to entangle the qubits and rotations around all 3 axes for calculations

is the Controlled NOT gate (see the first 3 gates in Fig 2). This gate negates the target qubit (indicated with a $\oplus$ sign) if the control qubit (indicated with a $\bullet$ sign) is in state $|1\rangle$. By using this gate, entangled states can be created. Some quantum gates, such as the rotation gates, have parameters. A rotation gate rotates the quantum state around one of the axes on the Bloch sphere by an angle θ.

Measurements, which are the only non-unitary operations on a quantum circuit, are mostly placed at the end of a circuit (see the last operations in Fig 1). By measurement, classical information is harvested from a quantum system, and the qubits are collapsed to one of the computational basis states [23].

Variational quantum circuits (VQCs) are circuits with parameterized gates. Different values can be assigned to these parameters to obtain various final states. This type of circuits is used for classical-quantum hybrid algorithms such as QAOA [9] or VQE [24]. In these algorithms, a parameterized quantum circuit is run for different parameter values, and these values are optimized using a classical computer. For optimization, a range of optimizers, such as the Adam optimizer [33] or a genetic algorithm [5], can be used.

A VQC always starts with an encoding layer, followed by a calculation layer that can consist of one or more sublayers, and ends with a measurement (see Fig. 1). The encoding layer creates a quantum state based on the input data. The calculation layer performs parameterized operations and the measurement is used to extract classical information. Many structures are possible for the calculation layer as long as all qubits get entangled and parameterized operations are performed. An analysis of different types of VQCs was done in [32]. We will use a simple ansatz of multiple repetitions of the same

Method	Data	Qubits	Depth
Basis encoding	String of n bits	$O(n)$	$O(1)$
Angle encoding	n real values	$O(n)$	$O(1)$
Amplitude encoding	n real values	$O(\log(n))$	$O(n)$

Table 1: Complexity comparison of different encoding methods

block. The block will consist of an entanglement layer followed by a rotation layer. An example of such a block is given in Fig. 2.

2.3 Encoding

An important step in creating a hybrid algorithm is encoding [36] of classical data into a quantum state and interpreting the result of the quantum circuit. The chosen method of encoding affects the data we can encode, the number of qubits needed, and the depth of the encoding circuit (Table 1). Many possible encoding methods exist, and here we present the three most common ones. The simplest method of encoding classical data is binary encoding, which encodes one classical bit into one qubit [36], but this is often not feasible with the currently small number of qubits available, and we have to use a denser encoding.

Another possible encoding is angle encoding [36] which encodes one real value into a single qubit by using a rotation operator with the classical data as the rotation parameter. It is also possible to encode two classical values into a single qubit by using two different rotation gates as a single qubit has two degrees of freedom.

A possibly denser encoding is amplitude encoding [36] which encodes classical values into the amplitudes of the quantum states and allows the encoding of 2^n values in n qubits. While binary encoding and angle encoding are simple and only require one gate for each qubit, amplitude encoding is more complex and scales exponentially with the number of qubits, and thus, is linear with the number of classical values.

For both angle and amplitude encoding, the input data has to be normalized. As the rotation gates are periodic with a period of 4π, the input has to be in an interval of length less than 4π to assure that the encoding is injective. Often a smaller interval like $[0, \pi]$ or $[0, 2\pi]$ is used. For this scaling, the interval of possible input values has to be known. For amplitude encoding, the amplitudes have to be a normalized vector, and thus, the input data vector has to be normalized.

Another issue is the decoding of the result into classical data. Because of its non-deterministic nature, a quantum state randomly collapses into one of its 2^n possible states after a measurement. This randomness can be resolved by running the circuit multiple times and using the most common result.

Another way is not to use the resulting bit string as a result, but instead, use the probabilities as an output. This will create n real values in the interval $[0, 1]$ instead of n bits. The probability values can be approximated by running the circuit multiple times and the approximation will get better with more measurements. In a simulator, it is not necessary to run the circuit multiple times as the probability values can be accessed directly. Hence, our proposed approach in Section 4 can be run not only as a quantum algorithm exploiting the principles of quantum mechanics and running on quantum computers, but also as a quantum-inspired algorithm [19] running only on classical computers in an efficient way. This is because the VQCs used in our approach are not complex and fast to simulate.

The choice of encoding and decoding is important because it affects the number of qubits required, and the number of qubits is still a limiting factor for current quantum computer. Also the viability of classical simulation is limited by the number of qubits.

3 OUR APPROACH: USING QUANTUM MACHINE LEARNING FOR JOIN ORDER OPTIMIZATION

An important task in a DBMS is the optimization of join orders. For a low number of tables, dynamic programming can be used for join order optimization, but this is not possible for a larger number of tables as the number of possible orders grows exponentially. For n tables and ignoring the ordering of a single join pair, the number of join orders is given by $\frac{(2(n-1))!}{2^{n-1}(n-1)!}$[1]. This problem can be eliminated by employing machine learning to predict an efficient join order from the knowledge learned from past join orders. In this section, we propose an approach that uses quantum computing to implement machine learning for join order optimization.

Many great advances in machine learning were made using neural networks, which were proven to be universal approximators [14]. It is possible to design a quantum algorithm that can mimic the learning behavior of a neural network by replacing it with a VQC. In this type of approach, learning is achieved by adjusting the parameters of the quantum gates on the circuit similar to adjusting the weights of the connections between the nodes of a neural network.

3.1 Encoding the join order problem

To use machine learning, we have to create a numeric representation of a query. Many representations exist for a database with m tables like a $m \times m$ matrix [39] or a vector of m rational values [21]. As the number of qubits in current quantum computers is small, we require a representation that uses as few values as possible. We will instead represent a join of n tables as a vector of n integers by assigning an integer ID to each table in the database. For our quantum circuit, we will encode these n values into n qubits using angle encoding. We choose angle encoding because it requires fewer qubits than basis encoding and can be done with a single rotation gate for each qubit. For this, we will map the n different IDs x from the integer interval $[0, n-1]$ to a feature y in the interval $[0, \pi]$ with $y = \frac{x\pi}{n-1}$. We choose this interval as it maps the extremes of the interval to orthogonal states. If we use the R_x gate, it will map the base state $|0\rangle$ to $R_x(0) \cdot |0\rangle = |0\rangle$ and $R_x(\pi) \cdot |0\rangle = |1\rangle$

The number of required qubits also depends on the number of possible outputs we require. There are two ways to generate a join order with machine learning. One way is to generate the complete join order in a single step. As the output is a complete join order, the number of outputs scales with the number of possible join orders

[1] According to [25] the number of join orders is $\frac{(2(n-1))!}{(n-1)!}$ without ignoring the ordering of a single join pair.

Tables	Single Step		Multi Step	
	Join orders	Qubits	Joins	Qubits
3	3	2	3	2
4	15	4	6	3
5	105	7	10	4
10	34 459 425	26	46	6
75	$4.09 \cdot 10^{128}$	443	2775	12
n	$\frac{(2(n-1))!}{2^{n-1}(n-1)!}$	$\lceil \log_2(\frac{(2(n-1))!}{2^{n-1}(n-1)!}) \rceil$	$\binom{n}{2}$	$\lceil \log_2(\binom{n}{2}) \rceil$

Table 2: Comparison between the single-step and multi-step approaches in join order generation. With the current best quantum computer (IBM Osprey) having 443 qubits, up to 75 tables could be joined in a single step.

as $\frac{(2(n-1))!}{2^{n-1}(n-1)!}$. The other possibility is generating a join order by joining two tables in each step. In this case, the number of outputs is the number of possible combinations of two tables $\binom{n}{2}$, but in turn, the generation takes $n-1$ steps. As we can enumerate all possible joins, we can assign a quantum state to each join and use the quantum state with the highest probability as the chosen join order. The number of required qubits is the logarithm of the number of required outputs. Table 2 shows the number of the outputs and required qubits for different numbers of tables in the query.

3.2 Learning

As we want to learn to predict the best join order, we have to optimize the parameters of our VQCs. For this, we have to define a measurement of the quality of the current parameter. One way to do this would be the classification approach, where we define a correct join order for every query and measure the quality by the number of correctly chosen join orders. Another possibility is to define a reward function that measures the quality of the chosen join orders as a real value instead of a binary choice between right and wrong as it is done in reinforcement learning [17]. The latter works better because there exists no binary choice between a right order and a wrong order as a join order will sometimes be only slightly worse. We can learn to predict the best join order using the reward only as a measure of the quality of our learning, but we can also learn to predict the reward itself. If we can predict the reward, we can choose the join order with the highest predicted reward.

If we encode a join order in each quantum state, we can interpret the corresponding amplitude as the reward. As the amplitude vector has to be a unit vector, the VQC can not predict arbitrary values for the reward. If we define the reward as a value in the interval $[0, 1]$ and scale the amplitude to this interval, at least one join order will have the maximal reward of 1 and is chosen as the best. The rest of the rewards are relative to the best.

To optimize our model, we have to define an error between the predicted rewards p and the actual rewards r for n join orders. For each join order i we have the predicted reward p_i and the actual reward r_i and we use the sum of the squared errors between these:

$$l = \sum_{i=0}^{n-1} \left(\frac{p_i}{\max p} - r_i \right)^2 \tag{2}$$

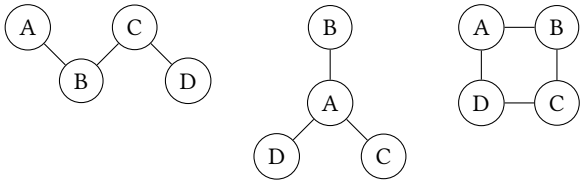

Figure 3: Three different query structures for 4 relations: Chain join (left), Star Join (middle), Ring join (right)

The algorithm learns by minimizing this error function. As this function is differentiable, we can use gradient-based optimizers.

3.3 Cross Joins

A cross join is a join for which no join condition exists and will result in many results as it returns the Cartesian product of the two relations. Even for a query that can be executed without requiring a cross join, some join orders will result in a cross join. As a cross join greatly increases the number of intermediate results, it is generally worse than a join order without a cross join.

As our algorithm can return every possible join order as a result, which includes join orders that require a cross joins. A possibility for improvement is to prevent the generating of join orders with cross joins. A simple approach would be to allow cross joins in training but prevent them in execution. As our algorithm provides the predicted rewards for all join orders, we can sort the join orders by predicted reward. Then we chose the join order with the highest reward that doesn't require a cross join.

Another possibility is to filter the cross joins out of the training data before training and leave only the orders which require no cross join as possible choices. The number of these valid join orders depends on the structure of the query (see Figure 3). For 4 relations we have 5 valid join orders if it is a chain join, 6 if it is a star join and 10 if it is a ring join. For a query to form a ring, it has to have more join conditions than joins.

We now have a different number of valid outputs, even for the same number of inputs, dependent on the query. However, we want to use the same circuit for each query, which always produces the same number of n outputs. To generate a variable number of valid outputs, we need an interpretation that selects one s of the k valid outputs from the n predictions p and calculates a loss l between the n predictions p and the k correct outputs r.

Option 1: Ignore all except the first k values.

$$s = \arg \max_{x \in [0,k-1]} p_x$$

$$l = \sum_{i=0}^{k-1} \left(\frac{p_i}{\max p} - r_i \right)^2$$

Option 2: Assume $r_i = r_{i+k}$. For the chosen join order, we can use the modulo operator:

$$s = (\arg \max_x p_x) \mod k$$

For the loss, we want to be each join order in the same congruence class:

$$l = \sum_{i=0}^{k-1} \left(\frac{p_i}{\max p} - r_i \mod k \right)^2$$

Option 3: Map the n values p to k values q. As we now have k values, both selection s and loss l can be easily defined as:

$$s = \arg\max_x p_x$$

$$l = \sum_{i=0}^{k-1} \left(\frac{q_i}{\max q} - r_i \right)^2$$

The mapping from p to q can be defined in many ways. We experiment with the following three mappings with $P_i = \{p_{i+xk} | x \in \mathbb{N}\}$:
a) *ModSum*: Using the sum of elements: $q_i = \sum_x p_{i+xk}$
b) *ModMax*: Using the maximum: $q_i = \max P_i$
c) *ModMean*: Using the average of the elements: $q_i = \frac{\sum_{x \in P_i} x}{|P_i|}$

4 IMPLEMENTATION DETAILS

We implement a single-step join order generation of an arbitrary number of tables, but we use only 4 tables in our experiments since for this number, the required qubits for input and output are equal and we only have one invalid output state. In the experiments (see Section 5), we show that our approach can compete with classical heuristics like the built-in genetic algorithm of PostgreSQL when joining 4 tables. With the addition of cross join elimination our approach can even outperform these approaches. As a reward function, we use $r = \frac{b}{c}$, with b being the cost of the best and c of the chosen join order. This reward function has an intuitive interpretation, that a reward with the value $\frac{1}{x}$ means that the chosen join c order has a x times higher cost than the best possible b. Join orders that are only a few percent worse than the best will still receive a high reward, while join orders that are many times worse will get a very low reward. One possible cost for join orders is the number of intermediate results, and a join order is better if it generates fewer intermediate results. Another possible cost function is the execution time of the query, with a lower execution time being better. An advantage of using the number of intermediate results is that it is independent of the hardware and can be exactly measured, but it does not consider the costs of different join methods, which is relevant in a DBMS. Because of this, we will use the execution time as the cost as it is most relevant in a real application.

The parameters of our model are the angles of the rotation gate, which we optimize using stochastic gradient descent (SGD). The gradients are computed using the parameter shift rule [30].

We have used Python (v. 3.8.12) and the Qiskit library (v. 0.19.1) for simulating our quantum circuits. Additionally, we have used the library PyTorch (v. 1.11.0) for machine learning, which allows neural networks to be easily created and optimized. With its Torch Connector, Qiskit can easily be connected with PyTorch to create hybrid algorithms using the PyTorch implementation of SGD.

We also developed a plugin[2] for PostgreSQL which allows the direct integration of a trained VQC model as a join order optimizer in PostgreSQL. This can easily be done by using PostgreSQL's plugin and hook system. The hook system allows us to replace the *standard_join_search* with our own function, which will send it to a python script running our model using the trained parameters. By only replacing the join order optimizer, we still support the full SQL syntax, as everything except the join order is still handled by PostgreSQL.

5 EVALUATION

For our benchmark, we use the ErgastF1[3] dataset, which contains information about the Formula One series. We use this dataset because it has enough tables to create many possible joins while at the same time being small. The dataset has to be large enough so that different join orders have a significant difference in execution time, while also being small enough that a benchmark of all possible joins can be created in a reasonable time. The dataset contains 13 tables with 19 relations between them, from which we can create 185 different sets of 4 tables to join. We create a query for each set of tables that can be joined without requiring a cross join. These queries contain the primary key-foreign key relations as join conditions, but no other filters and use SELECT * to return all results.

We benchmark these queries using PostgreSQL (v. 14.4) and the EXPLAIN ANALYZE SQL operation to get the execution time, which we store for each possible join order for each query. We use this benchmark to train our model. While the training is done independently of PostgreSQL, the trained model can be integrated into PostgreSQL where it replaces the standard join order optimizer.

For the evaluation of the trained model, we use our developed plugin. This way the method of measurement of our QML model is identical to the measurement of the dynamic programming (DP) and genetic algorithm (GEQO) as the difference only occurs inside PostgreSQL's planner module. We consider only the execution time of a query without the planning time to avoid any disadvantage of the QML occurring from the simulation time.

The optimizer runs for 8000 iterations and in each iteration, a random set of tables is selected and given as input. The optimizer uses a decaying learning rate to achieve a stable final state for the model parameters. The learning rate starts at 0.01 and is reduced by a factor of 0.9 every 100 iterations. Every 100 iterations, the quality of the model is measured by calculating the average reward over all possible queries.

Figure 4 shows the evolution of the average reward over the runtime of the optimizer, as well as the average reward DP and GEQO would produce. Even though DP finds the theoretically best solution according to the cost estimation, it only reaches a reward of 0.836. This is caused by the inaccuracy of the cost estimation and as both DP and GEQO work with the cost estimation, their quality is limited by it. As our QML approach does not rely on cost estimation, it has a chance to be better than it. We can see that our VQC with 20 layers is better than GEQO, but worse than DP. Our extended approach ModMax reaches a better reward than DP. We can also see that it has a relatively high reward from the beginning as the limitation to only join orders without a cross join prevents it from choosing the worst join orders

[2]We published our QML optimizer integrated into a PostgreSQL plugin on GitHub: https:github.comTobiasWinkerQC4DB_QML

[3]http://ergast.com/mrd/

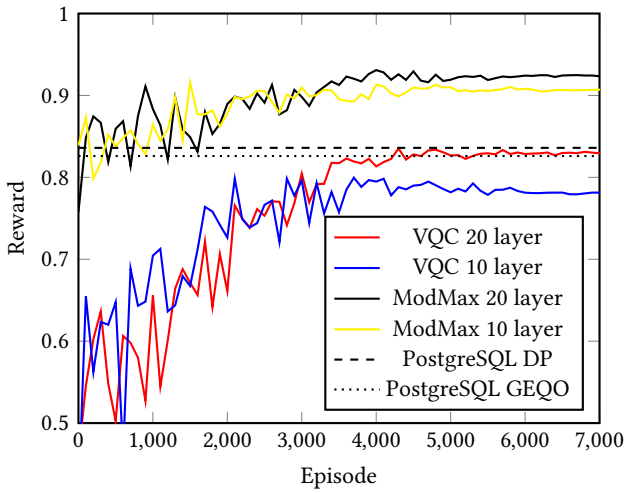

Figure 4: Average rewards of the VQC with and without cross joins in the training data. For the model without cross joins we use Option 3b ModMax from section 3.3

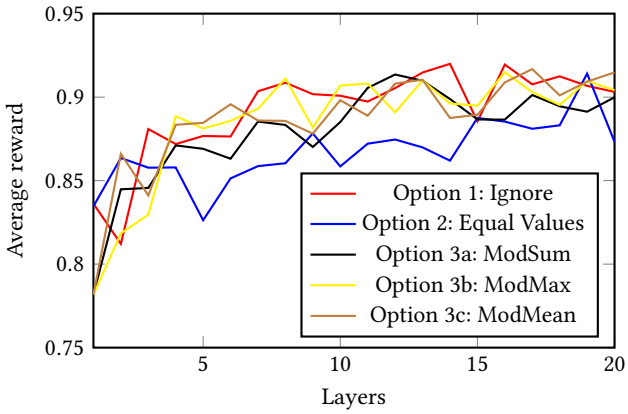

Figure 5: Reward for the final model for different number of layers and methods of eliminating cross joins as discussed in section 3.3

In figure 6 we look at the total runtime of our set of queries. We also include a comparison with the classical approach RTOS [1, 40] as well as our VQC with cross joins filtered out only in execution. With the addition of this filter, our VQC reaches a similar quality as the classical DP. Our final QML model ModMax see section 3.3, which filters the cross joins before training, is 2.7% faster than DP, but still 6.0% slower than RTOS. Even though the total runtime of RTOS is better, the average reward for the individual queries is only 0.879, i.e., 4.9% worse than ModMax.

In figure 5 we compare the quality of the different options proposed in section 3.3 for different numbers of layers. We can see that the quality improves with the number of layers, but we do not get a smooth curve, as the final result also depends on the randomly chosen initial parameters. We see that option 1 and all variants of

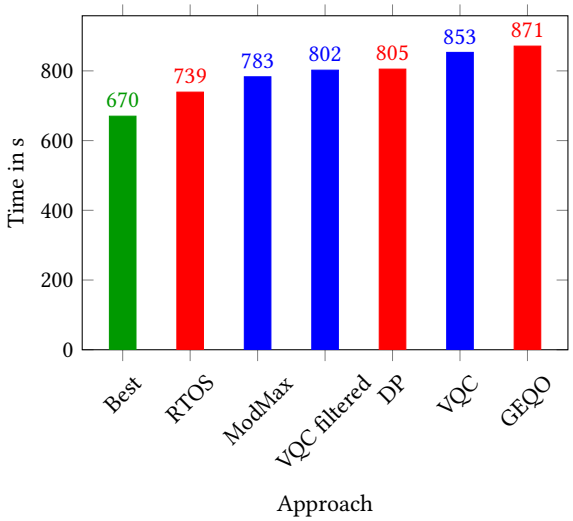

Figure 6: Comparison of the combined runtime of all queries in the benchmark for different optimization methods. *Best* is the optimal join order found by trying all possible join orders. *DP* and *GEQO* are the integrate optimizers of PostgreSQL. *VQC* is a 20 layer VQC. *VQC filtered* is the same circuit, but cross joins are filtered out in selection. ModMax is a VQC trained with the modification of section 3.3

option 3 have similar results. Only option 2 is visibly worse than the others. This is expected as the assumption of equal values, imposes additional restrictions on the model, which have to be learned.

6 CONCLUSIONS

In this paper, we have shown that a hybrid quantum algorithm can be used to solve the join order optimization problem. This result is obtained by using variational quantum circuits (VQCs). For this approach, we developed an encoding for the possible join operations and a decoding to obtain the join orders from the results of the proposed circuits. We have also demonstrated using a widely accepted benchmark procedure that simple VQCs perform similarly to classical optimization methods and can outperform them if improved by cross join elimination.

The experiments we conducted are for joining 4 tables, which require only 4 qubits to be optimized. Quantum computers with these many qubits are already available. One of the strong sides of the approach is that it only requires a number of qubits sufficient to represent all possible solutions. There is no need for any additional qubits. This makes optimizing the join order of a practical number of tables possible using contemporary quantum computers. For instance, using the VQC approach to optimize the join order of 10 tables would require 26 qubits.

We used a relatively simple VQC to show that it can compete with classical optimizers. By using a more complex circuit with more gates and qubits, the resulting quality produced by the VQC could still be improved. Other components of our approach like the encoding and decoding, the reward function, the loss function and the choice of the optimizer could also be adapted to achieve better

results. Our developed code allows the replacement of all of these components. We have only used simulators to run our quantum circuits. Our future work includes extensive evaluations on real quantum computers.

ACKNOWLEDGMENTS

This work is funded by the German Federal Ministry of Education and Research within the funding program quantum technologies - from basic research to market – contract number 13N16090.

REFERENCES

[1] [n. d.]. Reinforcement Learning with Tree-LSTM for Join Order Selection. https://github.com/TsinghuaDatabaseGroup/AI4DBCode/tree/master/RTOS. Commit bcb1fa8cab96b835138435151ed126c391d6a3d3.

[2] Mert Akdere, Ugur Çetintemel, Matteo Riondato, Eli Upfal, and Stanley B Zdonik. 2012. Learning-based query performance modeling and prediction. In *2012 IEEE 28th International Conference on Data Engineering*. IEEE, 390–401.

[3] Matthias C Caro, Hsin-Yuan Huang, M Cerezo, Kunal Sharma, Andrew Sornborger, Lukasz Cincio, and Patrick J Coles. 2022. Generalization in quantum machine learning from few training data. *Nat. Commun.* 13, 1 (2022).

[4] Jin Chen, Guanyu Ye, Yan Zhao, Shuncheng Liu, Liwei Deng, Xu Chen, Rui Zhou, and Kai Zheng. 2022. Efficient Join Order Selection Learning with Graph-Based Representation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Washington DC, USA) *(KDD '22)*. Association for Computing Machinery, New York, NY, USA, 97–107. https://doi.org/10.1145/3534678.3539303

[5] Samuel Yen-Chi Chen, Chih-Min Huang, Chia-Wei Hsing, Hsi-Sheng Goan, and Ying-Jer Kao. 2022. Variational quantum reinforcement learning via evolutionary optimization. *Machine Learning: Science and Technology* 3, 1 (2 2022), 015025. https://doi.org/10.1088/2632-2153/ac4559

[6] Samuel Yen-Chi Chen, Chao-Han Huck Yang, Jun Qi, Pin-Yu Chen, Xiaoli Ma, and Hsi-Sheng Goan. 2020. Variational Quantum Circuits for Deep Reinforcement Learning. *IEEE Access* 8 (2020), 141007–141024. https://doi.org/10.1109/ACCESS.2020.3010470

[7] Yuxuan Du, Min-Hsiu Hsieh, Tongliang Liu, and Dacheng Tao. 2020. Expressive power of parametrized quantum circuits. *Physical Review Research* 2, 3 (2020), 033125.

[8] Tobias Fankhauser, Marc E. Solèr, Rudolf M. Füchslin, and Kurt Stockinger. 2021. Multiple Query Optimization using a Hybrid Approach of Classical and Quantum Computing. https://doi.org/10.48550/ARXIV.2107.10508

[9] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028* (2014).

[10] Archana Ganapathi, Harumi Kuno, Umeshwar Dayal, Janet L Wiener, Armando Fox, Michael Jordan, and David Patterson. 2009. Predicting multiple metrics for queries: Better decisions enabled by machine learning. In *2009 IEEE 25th International Conference on Data Engineering*. IEEE, 592–603.

[11] Chetan Gupta, Abhay Mehta, and Umeshwar Dayal. 2008. PQR: Predicting query execution times for autonomous workload management. In *2008 International Conference on Autonomic Computing*. IEEE, 13–22.

[12] Rakebul Hasan and Fabien Gandon. 2014. A machine learning approach to sparql query performance prediction. In *2014 IEEE/WIC/ACM International Joint Conferences on Web Intelligence (WI) and Intelligent Agent Technologies (IAT)*, Vol. 1. IEEE, 266–273.

[13] Jonas Heitz and Kurt Stockinger. 2019. Join Query Optimization with Deep Reinforcement Learning Algorithms. *arXiv:1911.11689* (2019).

[14] K. Hornik, M. Stinchcombe, and H. White. 1989. Multilayer feedforward networks are universal approximators. *Neural Networks* 2, 5 (1989), 359–366.

[15] Hsin-Yuan Huang, Michael Broughton, Masoud Mohseni, Ryan Babbush, Sergio Boixo, Hartmut Neven, and Jarrod R McClean. 2021. Power of data in quantum machine learning. *Nat. Commun.* 12, 1 (2021).

[16] Kyoungmin Kim, Jisung Jung, In Seo, Wook-Shin Han, Kangwoo Choi, and Jaehyok Chong. 2022. Learned cardinality estimation: An in-depth study. In *Proceedings of the 2022 International Conference on Management of Data*. 1214–1227.

[17] Yuxi Li. 2017. Deep reinforcement learning: An overview. *arXiv preprint arXiv:1701.07274* (2017).

[18] Owen Lockwood and Mei Si. 2020. Reinforcement Learning with Quantum Variational Circuits. https://doi.org/10.48550/ARXIV.2008.07524

[19] A. Manju and M. J. Nigam. 2014. Applications of Quantum Inspired Computational Intelligence: A Survey. *Artif. Intell. Rev.* 42, 1 (2014), 79–156. https://doi.org/10.1007/s10462-012-9330-6

[20] Ryan Marcus and Olga Papaemmanouil. 2018. Deep Reinforcement Learning for Join Order Enumeration. In *Proceedings of the First International Workshop on Exploiting Artificial Intelligence Techniques for Data Management* (Houston, TX, USA) *(aiDM'18)*. Association for Computing Machinery, New York, NY, USA, Article 3, 4 pages. https://doi.org/10.1145/3211954.3211957

[21] Ryan Marcus and Olga Papaemmanouil. 2018. Deep Reinforcement Learning for Join Order Enumeration *(aiDM'18)*. Association for Computing Machinery, New York, NY, USA, Article 3, 4 pages. https://doi.org/10.1145/3211954.3211957

[22] Nitin Nayak, Jan Rehfeld, Tobias Winker, Benjamin Warnke, Umut Çalikyilmaz, and Sven Groppe. 2023. Constructing Optimal Bushy Join Trees by Solving QUBO Problems on Quantum Hardware and Simulators. In *Proceedings of the International Workshop on Big Data in Emergent Distributed Environments (BiDEDE), Seattle, WA, USA*. https://doi.org/10.1145/3579142.3594298

[23] Michael A Nielsen and Isaac Chuang. 2002. *Quantum computation and quantum information*. American Association of Physics Teachers.

[24] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J Love, Alán Aspuru-Guzik, and Jeremy L O'brien. 2014. A variational eigenvalue solver on a photonic quantum processor. *Nature communications* 5, 1 (2014), 1–7.

[25] Saeed K Rahimi and Frank S Haug. 2010. *Distributed database management systems*. Wiley-Blackwell.

[26] Raghu Ramakrishnan, Johannes Gehrke, and Johannes Gehrke. 2003. *Database management systems*. Vol. 3. McGraw-Hill New York.

[27] Patrick Rebentrost, Masoud Mohseni, and Seth Lloyd. 2014. Quantum Support Vector Machine for Big Data Classification. *Phys. Rev. Lett.* 113 (2014), 130503. Issue 13. https://doi.org/10.1103/PhysRevLett.113.130503

[28] JJ Sakurai and JJ Napolitano. 2014. *Modern Quantum Mechanics 2nd edn (Harlow*. Pearson.

[29] Manuel Schönberger. 2022. Applicability of Quantum Computing on Database Query Optimization *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 2512–2514. https://doi.org/10.1145/3514221.3520257

[30] Maria Schuld, Ville Bergholm, Christian Gogolin, Josh Izaac, and Nathan Killoran. 2019. Evaluating analytic gradients on quantum hardware. *Physical Review A* 99, 3 (mar 2019). https://doi.org/10.1103/physreva.99.032331

[31] P Griffiths Selinger, Morton M Astrahan, Donald D Chamberlin, Raymond A Lorie, and Thomas G Price. 1979. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*. 23–34.

[32] Sukin Sim, Peter D. Johnson, and Alá n Aspuru-Guzik. 2019. Expressibility and Entangling Capability of Parameterized Quantum Circuits for Hybrid Quantum-Classical Algorithms. *Advanced Quantum Technologies* 2, 12 (2019), 1900070. https://doi.org/10.1002/qute.201900070

[33] James Stokes, Josh Izaac, Nathan Killoran, and Giuseppe Carleo. 2020. Quantum Natural Gradient. *Quantum* 4 (5 2020), 269. https://doi.org/10.22331/q-2020-05-25-269

[34] Immanuel Trummer and Christoph Koch. 2016. Multiple Query Optimization on the D-Wave 2X Adiabatic Quantum Computer. *Proc. VLDB Endow.* 9, 9 (5 2016), 648–659. https://doi.org/10.14778/2947618.2947621

[35] Hongzhi Wang, Zhixin Qi, Lei Zheng, Yun Feng, Junfei Ouyang, Haoqi Zhang, Xiangxi Zhang, Ziming Shen, and Shirong Liu. 2020. April: An Automatic Graph Data Management System Based on Reinforcement Learning. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management*. 3465–3468.

[36] Manuela Weigold, Johanna Barzen, Frank Leymann, and Marie Salm. 2021. Encoding patterns for quantum algorithms. *IET Quantum Communication* 2, 4 (2021), 141–152. https://doi.org/10.1049/qtc2.12032

[37] Lucas Woltmann, Dominik Olwig, Claudio Hartmann, Dirk Habich, and Wolfgang Lehner. 2021. PostCENN: postgresql with machine learning models for cardinality estimation. *Proceedings of the VLDB Endowment* 14, 12 (2021), 2715–2718.

[38] Xiang Yu, Guoliang Li, Chengliang Chai, and Nan Tang. 2020. Reinforcement Learning with Tree-LSTM for Join Order Selection. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. 1297–1308.

[39] Xiang Yu, Guoliang Li, Chengliang Chai, and Nan Tang. 2020. Reinforcement learning with tree-lstm for join order selection. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 1297–1308.

[40] Xiang Yu, Guoliang Li, Chengliang Chai, and Nan Tang. 2020. Reinforcement learning with tree-lstm for join order selection. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 1297–1308.

[41] Kangfei Zhao, Jeffrey Xu Yu, Zongyan He, Rui Li, and Hao Zhang. 2022. Lightweight and Accurate Cardinality Estimation by Neural Network Gaussian Process. In *Proceedings of the 2022 International Conference on Management of Data*. 973–987.